Kubernetes and Istio

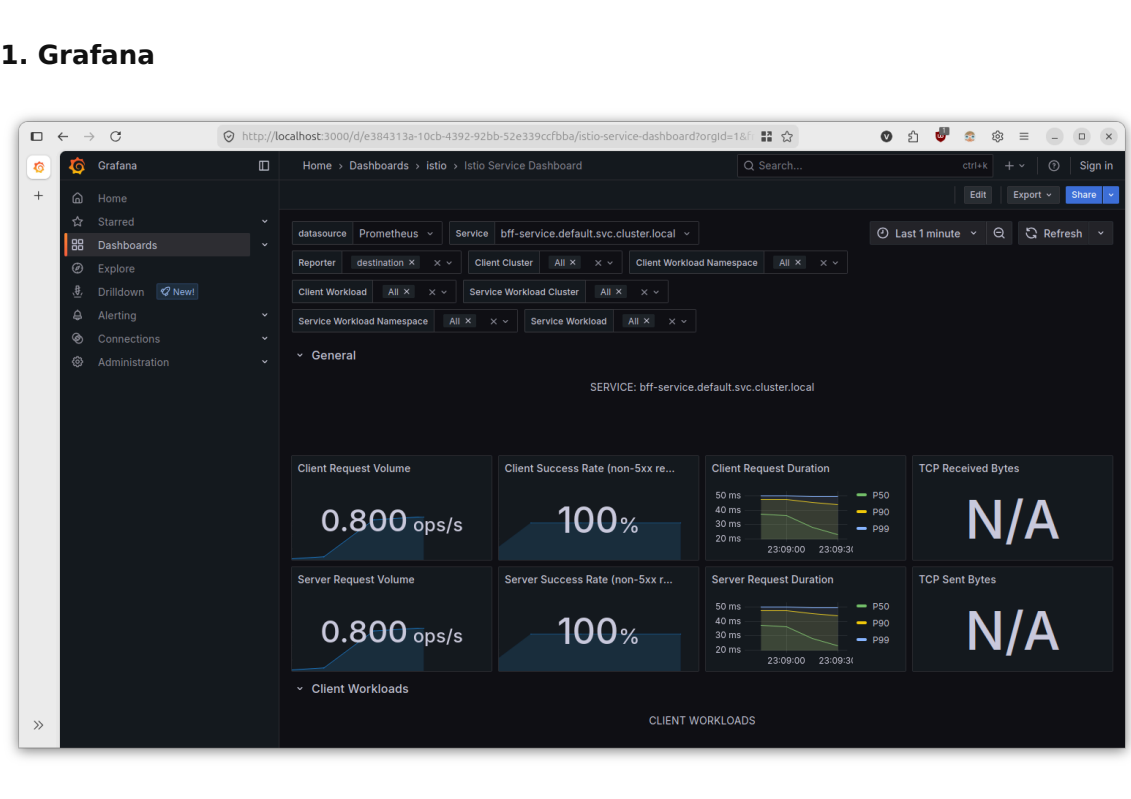
10 - Istio observability

Generated: June 2026
Version 1.0

10 - Istio observability

In a microservices architecture, tracking requests as they traverse multiple network boundaries is notoriously difficult. Istio solves this by injecting Envoy sidecar proxies next to your workloads. These proxies intercept all inbound and outbound traffic, allowing Istio to automatically generate rich telemetry-metrics, logs, and distributed traces-without requiring you to change your application code. Two of the most critical tools in the Istio ecosystem for visualizing this telemetry are **Grafana** (for *performance metrics*) and **Kiali** (for *usage metrics* and *topology*).

1. Grafana



datasource: Prometheus
Service: bff-service.default.svc.cluster.local
Reporter: destination
Client Cluster: All
Client Workload Namespace: All
Client Workload: All
Service Workload Cluster: All
Service Workload Namespace: All
Service Workload: All

SERVICE: bff-service.default.svc.cluster.local

Client Request Volume: 0.800 ops/s
Client Success Rate (non-5xx re...: 100%
Client Request Duration
TCP Received Bytes: N/A
Server Request Volume: 0.800 ops/s
Server Success Rate (non-5xx r...: 100%
Server Request Duration
TCP Sent Bytes: N/A

Grafana is an open-source analytics and interactive visualization web application. In the context of Istio, it is used to query and visualize the time-series metrics collected by Prometheus from the Envoy proxies.

> 🗒️ ***Installation***
> *Follow the guide at 8 - Istio # Run Grafana.*

What kind of metrics you can find

Istio exports standard metrics out-of-the-box that map directly to the Four Golden Signals of monitoring:

- **Traffic** (Volume): Measures the total number of requests (istio_requests_total).
- **Latency**: Measures the duration of requests (istio_request_duration_milliseconds).
- **Errors**: Tracks failed requests, usually segmented by HTTP status codes (e.g., 4xx and 5xx errors).
- **Saturation**: Tracks how "full" the service is (e.g., TCP connections, proxy CPU/memory usage).

When you install Grafana with Istio, you get several pre-configured dashboards:

- **Mesh Dashboard**: A high-level overview of the entire service mesh, showing global success rates and request volumes.
- **Service Dashboard**: Drills down into individual services (e.g., evaluating the performance of the checkout service).
- **Workload Dashboard**: Focuses on the underlying deployments or pods backing a service.

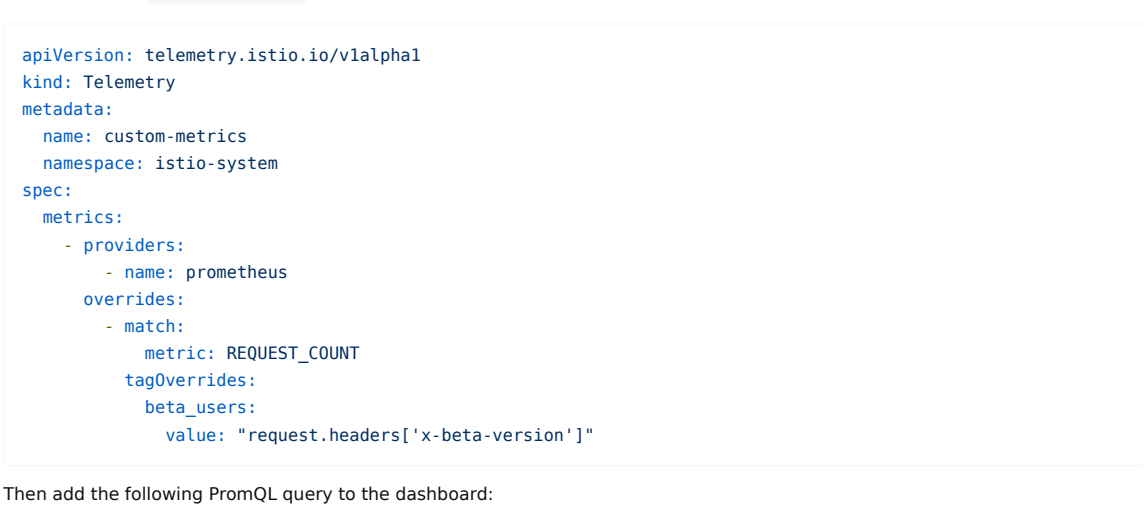
Add custom metrics

While standard metrics are powerful, you often need business-specific or infrastructure-specific dimensions (like tracking requests by a specific customer segment or custom HTTP header). Istio allows you to add custom metrics using the `Telemetry API`. You configure this by creating a Telemetry Custom Resource Definition (CRD). The following example demonstrates how to extract a custom header (`x-beta-version`) and append it as a new dimension to your metrics:

```yaml
apiVersion: telemetry.istio.io/v1alpha1
kind: Telemetry
metadata:
  name: custom-metrics
  namespace: istio-system
spec:
  metrics:
    - providers:
        - name: prometheus
      overrides:
        - match:
            metric: REQUEST_COUNT
          tagOverrides:
            beta_users:
              value: "request.headers['x-beta-version']"
```

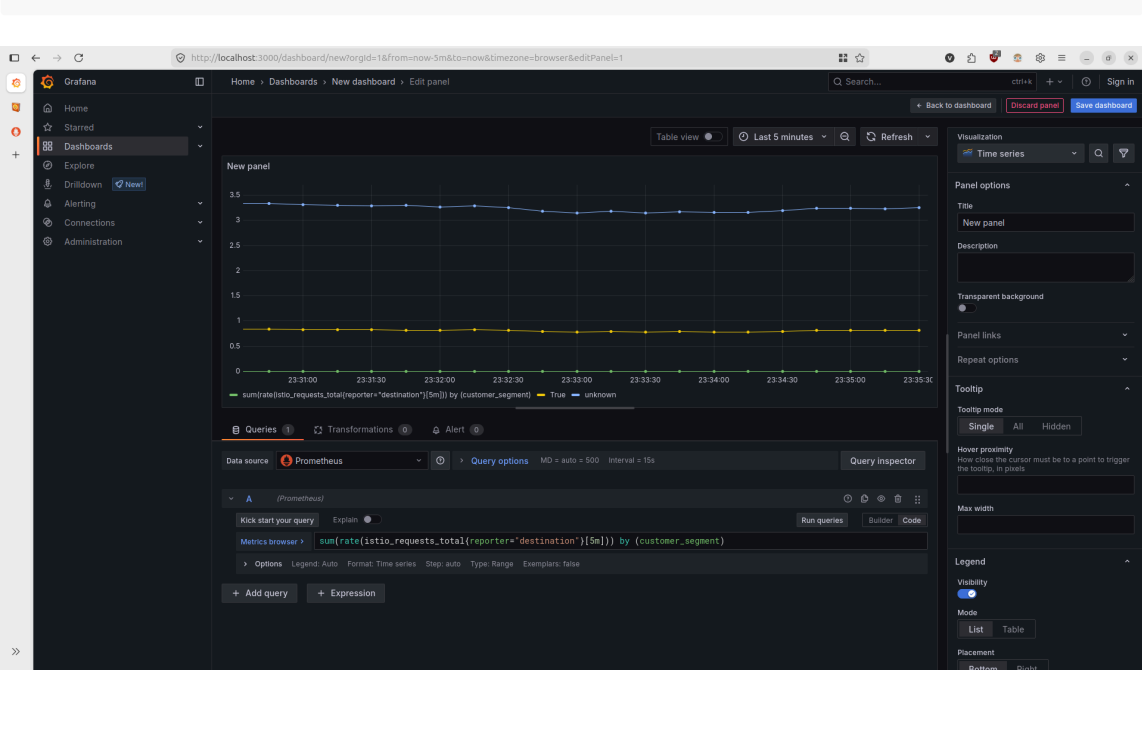
Then add the following PromQL query to the dashboard:

```
sum(rate(istio_requests_total{reporter="destination"}[5m])) by (customer_segment)
```



Data source: Prometheus
MD = auto = 500   Interval = 15s
A (Prometheus)
sum(rate(istio_requests_total{reporter="destination"}[5m])) by (customer_segment)
Legend: Auto   Format: Time series   Step: auto   Type: Range   Exemplars: false

2. Kiali



While Grafana excels at rendering time-series data as graphs and gauges, Kiali struggles to answer questions like: "*Which services are talking to each other?*" or "*Where is the traffic bottleneck in this specific transaction?*". Kiali is a management console specifically built for Istio to visualize service mesh topology and configuration health.

> 🗒️ ***Installation***
> *Run the following commands from the root of your downloaded Istio directory:*
>
> ```
> kubectl apply -f samples/addons/prometheus.yaml
> kubectl apply -f samples/addons/kiali.yaml
> ```
>
> *Once installed access the UI by port-forwarding.*
>
> ```
> istioctl dashboard kiali
> ```

What you can find

Kiali infers the structure of your microservices by analyzing the traffic flowing through them. It provides:

- **Topology Mapping (Service Graphs)**: Kiali auto-generates visual graphs of your architecture (App, Versioned App, Workload, and Service views). If a service starts failing, the corresponding node and its incoming edges turn red, allowing for immediate root-cause isolation.
- **Traffic Distribution & Shifting**: If you are running multiple versions of a service (e.g., v1 and v2), Kiali visually displays the exact percentage of traffic going to each. It even provides a wizard to implement traffic splitting directly from the UI.
- **Configuration Validation**: Kiali scans your Istio configuration objects (VirtualServices, DestinationRules, Gateways) and flags misconfigurations or orphaned routes before they cause an outage.
- **Security Visualization**: It visually indicates which edges are secured with mTLS (Mutual TLS) by displaying padlock icons on the connection lines.

> ❓ ***Why it is useful***
> *Modern microservice environments are too complex to map manually. Kiali bridges the gap between infrastructure configuration and real-time observability, turning raw Prometheus metrics into an actionable, live map of your distributed system.*